🎮 Exercício de Fixação – Joquempô Console com Ranking

Tema: Programação Orientada a Objetos com C# (.NET Console)

🎯 Objetivo

Desenvolver um sistema de jogo **Joquempô (Pedra, Papel, Tesoura)** no console, aplicando:

- Classes e encapsulamento
- Enumeradores (enum)
- Listas genéricas (`List<T>`)
- LINQ (Where / OrderByDescending)
- Expressões lambda
- Estruturas de repetição (`while`, `foreach`)
- Estruturas de decisão (`switch`, `if`)
- Tratamento de exceções
- Classe estática utilitária
- Uso de `new Random()` para simular o computador

Regra do jogo:

A pedra ganha da tesoura (amassa a tesoura).

A tesoura ganha do papel (corta o papel).

O papel ganha da pedra (embrulha a pedra).

🧩 Estrutura Geral

O sistema deve ter um **menu principal** com as opções:

1. Cadastrar jogador
2. Jogar
3. Exibir ranking
4. Sair

O menu deve funcionar dentro de um **laço `while`** e usar **`switch`** para tratar as opções.

🧱 Classes e responsabilidades

🧠 1. Classe `Jogador`

Função: representar um jogador do sistema.

Atributos:

- `Nome` (string)
- `Vitorias` (int)
- `Derrotas` (int)

Requisitos:

- Encapsular os atributos (`get` público, `set` privado).
- Criar um construtor que receba o nome do jogador.
- Criar métodos:
    - `void RegistrarVitoria()` → incrementa o número de vitórias.

- `void RegistrarDerrota()` → incrementa o número de derrotas.

✋ 2. Enum `Jogada`

Função: representar as opções possíveis do jogo.

Valores:

- `Pedra`
- `Papel`
- `Tesoura`

Uso:
O jogador escolhe um valor, e o computador gera outro valor aleatório com `new Random().Next(1, 4)`.

🎮 3. Classe `Partida`

Função: controlar uma partida entre um jogador e o computador.

Atributos:

- `Jogador` (referência ao objeto da classe Jogador)

Métodos:

- `Jogar()`:
  - Exibe as opções para o jogador escolher.
  - Lê a escolha do jogador (1, 2 ou 3).
  - Usa `new Random()` para gerar a jogada do computador.

- Mostra as duas jogadas na tela.
- Chama um método da classe `Utilitario` para decidir quem venceu.
- Atualiza o placar do jogador (vitória ou derrota).
- Deve usar **tratamento de exceções** (`try/catch`) para validar a entrada do usuário.

🧰 4. Classe estática `Utilitario`

Função: conter métodos auxiliares que não precisam de instância.

Métodos obrigatórios:

- `static string CalcularResultado(jogadaJogador, jogadaComputador)`
  - Retorna uma string com “Vitória”, “Derrota” ou “Empate” com base nas regras:
    - Pedra ganha de Tesoura
    - Tesoura ganha de Papel
    - Papel ganha de Pedra
- `static void Pausar()`
  - Aguarda o usuário pressionar uma tecla para continuar (usar Console.ReadKey).

🏆 5. Classe `Ranking`

Função: gerenciar a lista de jogadores e exibir o ranking.

Atributos:

- `List<Jogador>` para armazenar todos os jogadores cadastrados.

Métodos:

- `void AdicionarJogador(jogador)`
    - Adiciona um jogador à lista se ainda não existir outro com o mesmo nome.
- `Jogador BuscarJogador(nome)`
    - Retorna o jogador correspondente (usando LINQ e expressão lambda).
- `void ExibirRanking()`
    - Mostra os jogadores em ordem decrescente de vitórias (usando `OrderByDescending`).
    - Usa `foreach` para imprimir cada jogador e suas estatísticas.
    - Caso não haja jogadores, exibe uma mensagem informando isso.

🧭 6. Classe `Program` (principal)

Função: gerenciar o fluxo da aplicação e o menu interativo.

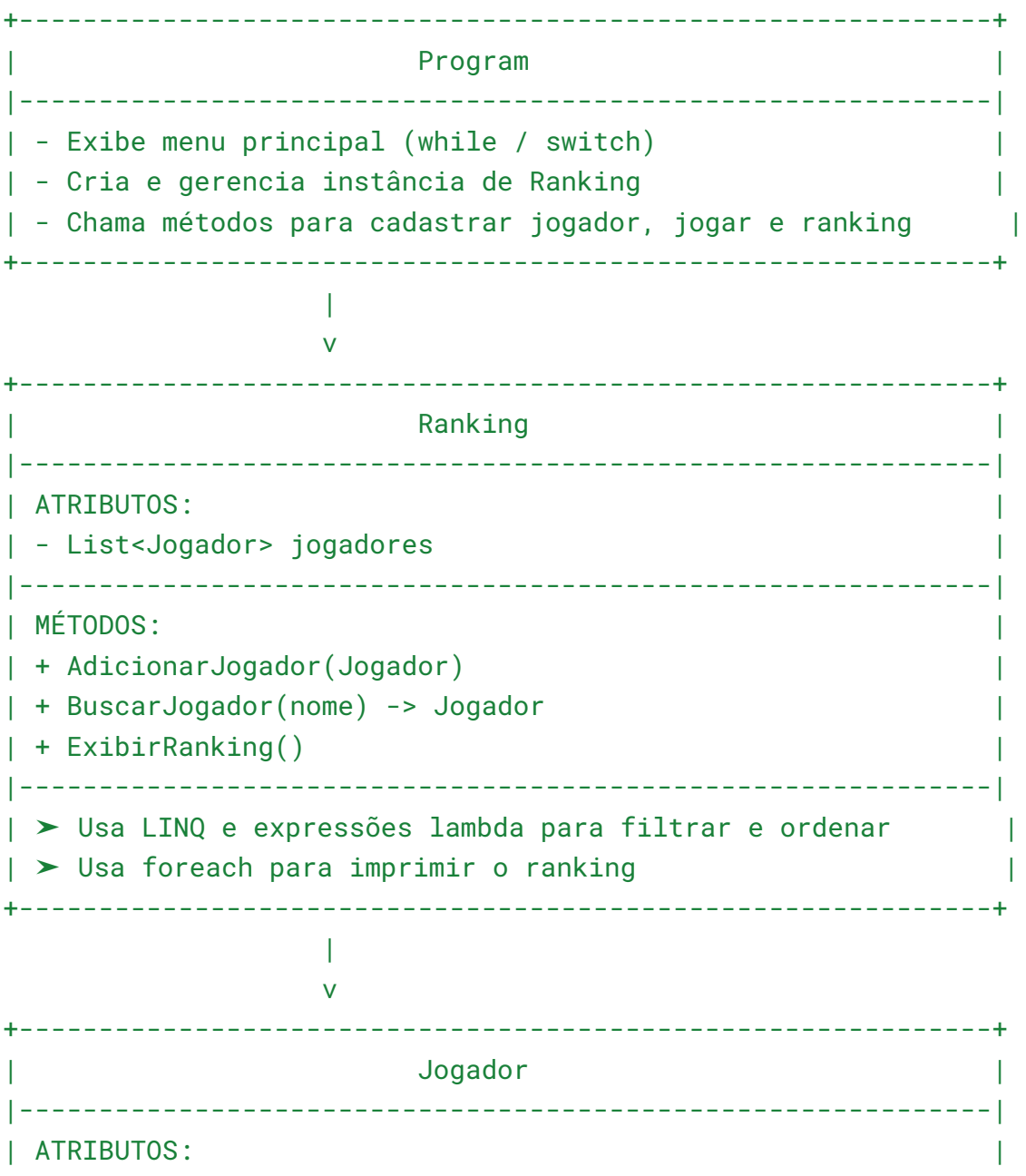
Responsabilidades:

- Criar uma instância do `Ranking`.
- Exibir o menu principal com as opções.
- Ler a entrada do usuário.
- Utilizar `switch` para tratar cada opção:
    - **Cadastrar jogador:** pede o nome e adiciona no ranking.
    - **Jogar:** pede o nome do jogador, busca no ranking e inicia uma nova `Partida`.
    - **Exibir ranking:** mostra a lista de jogadores com vitórias e derrotas.

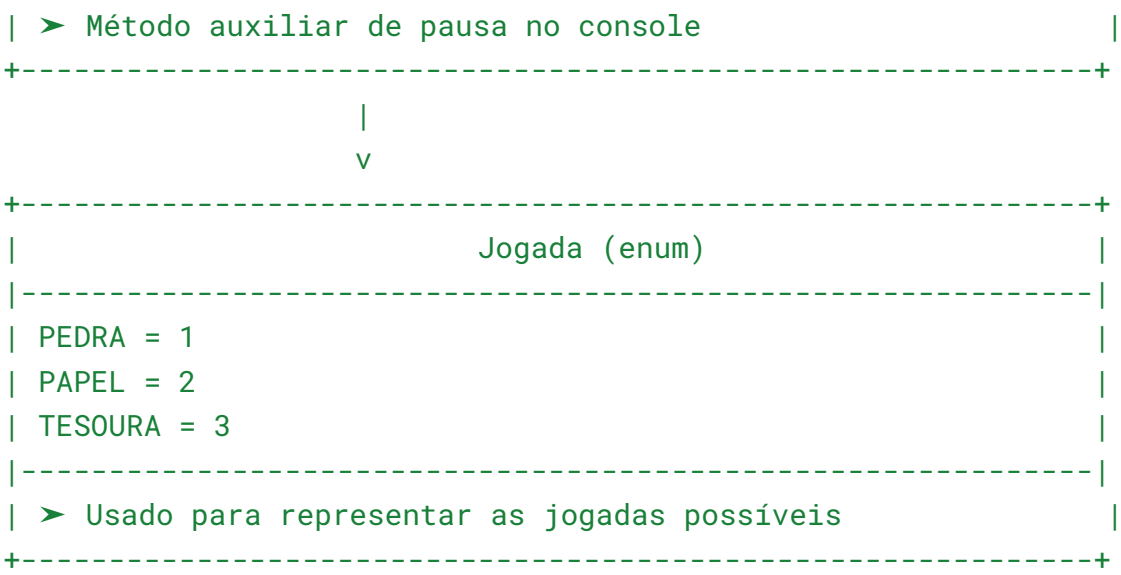
- **Sair:** encerra o programa.

- O menu deve repetir até o usuário escolher “0”.

🗺️ Mapa de Classes – Projeto Joquempô Console




- string Nome (somente leitura)
- int Vitorias (somente leitura)
- int Derrotas (somente leitura)
MÉTODOS:
+ RegistrarVitoria()
+ RegistrarDerrota()
➤ Representa o jogador humano
➤ Encapsulamento com propriedades privadas
Partida
ATRIBUTOS:
- Jogador Jogador
MÉTODOS:
+ Jogar()
➤ Lê jogada do usuário (1–3)
➤ Gera jogada da máquina com new Random()
➤ Usa Utilitario.CalcularResultado() para definir vencedor
➤ Atualiza vitórias/derrotas
➤ Usa try/catch para tratar exceções
Utilitario (static)
MÉTODOS ESTÁTICOS:
+ CalcularResultado(Jogada, Jogada) -> string
+ Pausar()
➤ Define as regras de vitória/empate/derrota


➤ Método auxiliar de pausa no console
Jogada (enum)
PEDRA = 1
PAPEL = 2
TESOURA = 3
➤ Usado para representar as jogadas possíveis